

Sample NoSQL platform: Apache HBase

dr hab. inż. Maciej Grzenda, prof. uczelni

M.Grzenda@mini.pw.edu.pl

<http://www.mini.pw.edu.pl/~grzendam>

Politechnika Warszawska
Wydział Matematyki i Nauk Informacyjnych



**European
Funds**

Knowledge Education Development

**Warsaw University
of Technology**

European Union
European Social Fund



MSc program in Data Science has been developed
as a part of task 10 of the project
„NERW PW. Science - Education - Development - Cooperation”
co-funded by European Union from European Social Fund.

Objectives

- NoSQL lacks formal definition
- Under the umbrella term of NoSQL more than 200 platforms are listed, which rely on largely different solutions
- This presentation is focused on Apache HBase – one of the most popular NoSQL platforms developed to store Big Data
- The way Apache HBase can be accessed to store and query data is discussed and compared with RDBMS solutions

Resources

1. [Chang2006] Chang F. et al., *Bigtable: A Distributed Storage System for Structured Data*, 2006, available at <http://research.google.com/archive/bigtable.html>
2. [HBASE] Apache HBase project website: <http://hbase.apache.org/>
3. [HBASEBook] Apache HBase TM Reference Guide, <https://hbase.apache.org/book.html>
4. [Kunigk2019] Kunigk, J. et al., *Architecting Modern Data Platforms. A Guide to Enterprise Hadoop at Scale*, O'Reilly, 2019
5. [Vohra2016], F. Vohra, *Apache HBase Primer*, Apress, 2016, ebook available via www.bg.pw.edu.pl

Apache HBase

- The leading database-related project in Hadoop group.
- Its goal is the hosting of very large tables - billions of rows X millions of columns - atop clusters of commodity hardware.
- is an open-source, distributed, versioned, column-oriented store modeled after Google's *Bigtable: A Distributed Storage System for Structured Data* by Chang et al. [Chang2006]
- Apache HBase provides Bigtable-like capabilities on top of Apache Hadoop and HDFS.
- Designed to be fault tolerant with particular emphasis on being tolerant to failures of cluster nodes
- Used by many big IT players (Facebook, Twitter, eBay, Yahoo!, ...) for managing large volumes of data including monitoring and performance data

For further details see <http://hadoop.apache.org/> and <http://hbase.apache.org/>

Hbase - applications

- ***When Should I Use HBase?***
- ***HBase isn't suitable for every problem.***
- *First, make sure you have enough data. If you have hundreds of millions or billions of rows, then HBase is a good candidate. If you only have a few thousand/million rows, then using a traditional RDBMS might be a better choice due to the fact that all of your data might wind up on a single node (or two) and the rest of the cluster may be sitting idle.*
- *Second, make sure you can live without all the extra features that an RDBMS provides (e.g., typed columns, secondary indexes, transactions, advanced query languages, etc.) An application built against an RDBMS cannot be "ported" to HBase by simply changing a JDBC driver, for example. Consider moving from an RDBMS to HBase as a complete redesign as opposed to a port.*
- *Third, make sure you have enough hardware. Even HDFS doesn't do well with anything less than 5 DataNodes (due to things such as HDFS block replication which has a default of 3), plus a NameNode.*

Cited from <http://hbase.apache.org/book/architecture.html#arch.overview>

Hbase and clusters

- HBase can run in:
 - Standalone mode,
 - Pseudodistributed i.e. based on one node-based cluster,
 - Fully distributed mode. In the latter case, a production quality cluster is supposed to contain at least 5 nodes.

Standalone is sufficient for learning purposes, unless physical data organisation and clustering options are supposed to be investigated

Hbase tables

- A table is a map of maps
- Every map is a key-value pair
- Keys are arbitrary strings that map to a row of data
- Row is a map with:
 - Keys being columns
 - Values being arbitrary uninterpreted arrays of values
- In addition:
 - Columns are grouped into column families
 - Hence columns can be referred with fully qualified names in the form 'family:columnInFamily'
 - In each row the list of used columns may be different

The number of column families is expected to be low i.e. at most three. See [HBASEBook] for details.

Hbase – sample table

- Let us assume we want to create a table of measurements
- Hence, let us define two column families:
 - **results** with columns **parameter,value,precision**
 - **time** with columns **timeStamp,timeResolution**
- For the beginning, we will create two rows:
 - Identified by keys '1' and '2'
 - A way to refer to a value of a parameter could be first/results:Category. This value could be e.g. 'Pressure'

Hbase – create table and put data

```
create 'measurements','results','time'
```

A table is created. No column names are declared, just column families

```
hbase(main):007:0> list
TABLE
measurements
wiki
wiki2
3 row(s) in 0.0560 seconds
```

List of tables is displayed

```
hbase(main):008:0> put 'measurements','1','results:parameter','pressure'
0 row(s) in 0.0330 seconds
```

New raw data is placed – one statement per a column value is needed. In this case, values of two columns from results column family are set

```
hbase(main):009:0> put 'measurements','1','results:value','17'
0 row(s) in 0.0200 seconds
```

```
hbase(main):010:0> scan 'measurements'
ROW          COLUMN+CELL
1            column=results:parameter, timestamp=1384201795340, value=pressure
1            column=results:value, timestamp=1384201798789, value=17
1 row(s) in 0.0220 seconds
```

The entire table content is displayed with scan.

```
hbase(main):011:0> get 'measurements','1','results:value'
COLUMN      CELL
results:value timestamp=1384201798789, value=17
1 row(s) in 0.0400 seconds
```

Alternatively one or more column values can be retrieved

```
hbase(main):012:0> get 'measurements','1','results:value','results:parameter'
COLUMN      CELL
results:parameter timestamp=1384201795340, value=pressure
results:value    timestamp=1384201798789, value=17
2 row(s) in 0.0450 seconds
```

HBase tables: discussion

- Only column families are declared when creating a table
- Whether a column family always contains the same columns in each row is fully up to a client application
- The benefit of having more than one column family is that every family may have its own settings (such as version settings)
- Since every row may contain even millions of columns, individual cells rather than entire rows are inserted and retrieved to/from HBase table
- Any cell includes an array of bytes from HBase perspective. No explicit support for different data types is provided.

Timestamp in HBase

- With every value, HBase stores timestamp, which is an integer value measuring the number of milliseconds since 1st January 1970 0:00 till the time of insertion
- The old value of the cell is not removed, but just left in a database with its own timestamp
- By default, the most recent version (in terms of its timestamp) of the data is returned
- It is possible to define a non-default timestamp at insertion time
- Also, more than one version of data can be returned
- HBase shell allows putting one value at a time. Hence, by default (when no explicit timestamps are specified), different columns values of a row get different time stamps. API can be used to insert entire row at the same time. In the latter case, all column values share the same timestamp.

```
hbase(main):012:0> get 'measurements','1','results:value','results:parameter'
COLUMN                                CELL
results:parameter                    timestamp=1384201795340, value=pressure
results:value                        timestamp=1384201798789, value=17
2 row(s) in 0.0450 seconds
```

Notice that each value of the same row in this case has its own, separate timestamp

Timestamp in practice

- The most recent version in a certain range can be requested
- Alternatively, the most recent k versions (here: $k=4$) can be requested

```
hbase(main):019:0> get 'measurements','5',{COLUMN=>'results:value',TIMERANGE=>[0,9999999999999999]}
COLUMN                                CELL
results:value                        timestamp=1384286927212, value=22.1
1 row(s) in 0.0660 seconds

hbase(main):020:0> get 'measurements','5',{COLUMN=>'results:value',TIMERANGE=>[0,9999999999999999],VERSIONS=>4}
COLUMN                                CELL
results:value                        timestamp=1384286927212, value=22.1
results:value                        timestamp=1384286910408, value=22.3
results:value                        timestamp=1384286896479, value=20
3 row(s) in 0.0640 seconds
```

Here three different versions of a cell have been returned. This is because Hbase keeps history of changes for every cell i.e. combination of row and column

Delete operation

- Can be performed at cell×tamp, cell, and row level

```
hbase(main):032:0> delete 'measurements','5','results:value',1384286910408
0 row(s) in 0.0150 seconds

hbase(main):033:0> get 'measurements','5',{COLUMN=>'results:value',TIMERANGE=>[0,9999999999999999],VERSIONS=>4}
COLUMN                                CELL
results:value                        timestamp=1384286927212, value=22.1
1 row(s) in 0.0650 seconds

hbase(main):034:0> delete 'measurements','5','results:value'
0 row(s) in 0.0200 seconds

hbase(main):035:0> get 'measurements','5',{COLUMN=>'results:value',TIMERANGE=>[0,9999999999999999],VERSIONS=>4}
COLUMN                                CELL
0 row(s) in 0.0320 seconds

hbase(main):036:0> get 'measurements','5'
COLUMN                                CELL
results:parameter                    timestamp=1384286901673, value=flow
1 row(s) in 0.0230 seconds

hbase(main):037:0> deleteall 'measurements','5'
0 row(s) in 0.0210 seconds

hbase(main):038:0> get 'measurements','5'
COLUMN                                CELL
0 row(s) in 0.0160 seconds
```

Accessing HBase platform and its tables

- Similarly to other NoSQL platforms, by default data in Hbase is NOT accessed via SQL statements
- Default way of accessing table data is through the use of Java libraries and by calling Java API
- Alternatively:
 - Requests can be sent over the network in REST model to perform actions such as reading row content, changing row or cell value
 - C/C++ client can be used
 - Scala can be used with HBase
- Additional project can be used to refer to HBase with SQL statements
- See [HBASEBook] for details

Summary

- Apache HBase is one of popular NoSQL platforms.
- It is used for Big Data storage and offers efficient fine-grained storage of large volumes of data.
- Table data are divided into rows, which:
 - are stored on many servers of a cluster
 - can include varied sets of columns in the same table
 - are frequently denormalised
 - can have a number of different versions for every cell they include
- Apache HBase:
 - handles other categories of data than RDBMS e.g. time series data
 - is usually used in combination with several other Big Data projects such as Apache Hadoop



**European
Funds**

Knowledge Education Development

**Warsaw University
of Technology**

European Union
European Social Fund



MSc program in Data Science has been developed
as a part of task 10 of the project
„NERW PW. Science - Education - Development - Cooperation”
co-funded by European Union from European Social Fund.